



Instruções:

Entrega parcial: 16/06/2026

Data limite de entrega: 30/06/2026

Data limite para apresentação: 03/07/2026

Trabalho em dupla

1) O que entregar: arquivo compactado cujo nome é “matricula1_nome1_matricula2_nome2”
(exemplo: 2011101001_caimi_2021100010_luciano) contendo:

1) Relatório incluindo:

- Introdução;
- Arquitetura proposta;
- Diagrama do datapath; (entrega parcial)
- Diagrama da FSM; (entrega parcial)
- Codificação de estados
- Tabela de transição de estados
- Tabela de saída
- Descrição dos módulos;
- Estratégia de testes;
- Resultados;
- Conclusões;
- Apendice A: detalhamento e formas de onda das simulações/testbenchs

2) Código fonte

3) Link para vídeo (incluir no relatório):

- explicando arquitetura implementada (datapath e controle);
- mostrando a execução no FPGA

2. Objetivos

O objetivo deste trabalho é aplicar os conceitos estudados ao longo da disciplina por meio do projeto e implementação de um sistema digital completo composto por:

- Datapath;
- Unidade de Controle;
- Máquina de Estados Finitos (FSM);
- Registradores;
- Contadores;
- Memória;
- Circuitos combinacionais;
- Projeto modular em Verilog.

Ao final do trabalho, os estudantes deverão ser capazes de especificar, implementar, testar e validar um sistema digital embarcado em FPGA.

3. Descrição do Problema

O trabalho consiste na implementação do jogo eletrônico Genius (Simon Game).

O sistema deverá gerar sequências luminosas que deverão ser reproduzidas pelo jogador através dos botões da placa FPGA.

A cada rodada:

1. Um novo símbolo é adicionado à sequência;
2. A sequência completa é exibida ao jogador;

3. O jogador reproduz a sequência utilizando os botões da placa;
4. O sistema verifica cada entrada;
5. Caso todas as entradas estejam corretas, uma nova rodada é iniciada;
6. Caso ocorra erro, a partida é encerrada.

O sistema deverá operar de forma totalmente autônoma após o acionamento do início.

4. Recursos da Placa DE1

A implementação deverá utilizar os seguintes recursos da placa.

4.1 Entradas

Recurso	Função
KEY0	Jogada 0
KEY1	Jogada 1
KEY2	Jogada 2
KEY3	Jogada 3
SW0	Iniciar partida
SW1	Reset

4.2 LEDs

Os LEDs vermelhos deverão representar os símbolos da sequência.

LED	Símbolo
LEDR0	Cor 0
LEDR1	Cor 1
LEDR2	Cor 2
LEDR3	Cor 3

Durante a exibição da sequência apenas um LED deverá permanecer aceso por vez.

4.3 Displays de Sete Segmentos

HEX0

Exibir o nível atual do jogador (valor entre 0 e F)

HEX3

Exibir o estado atual do sistema.

Exemplo:

Valor	Estado
0	IDLE
1	EXIBIÇÃO
2	ENTRADA
3	VITÓRIA
4	DERROTA

5. Regras do Jogo

O sistema utilizará quatro símbolos possíveis .

Cada símbolo será representado por um LED e por um botão correspondente.

A sequência inicial deverá possuir um único símbolo.

O jogo possui no total 15 níveis (ou rodadas). Caso o jogador complete a sequência do último nível com sucesso, o sistema transita para o estado WIN.

Após a exibição da sequência, o jogador terá um tempo máximo de 60 segundos para realizar cada rodada. Se o tempo expirar ou se errar a sequência, o jogo será direcionado para o estado LOSE.

A cada nova rodada:

- um novo símbolo é adicionado ao final da sequência;
- a sequência completa é exibida;
- o jogador deverá reproduzir toda a sequência.

O jogo termina quando:

- o jogador pressiona um botão incorreto;
- o jogador excede o tempo máximo permitido para resposta;
- o jogador alcança o nível máximo definido.

6. Arquitetura do Sistema

A solução deverá ser dividida obrigatoriamente em dois blocos:

Datapath:

Responsável pelo armazenamento e processamento das informações.

Unidade de Controle:

Responsável pela coordenação das operações do datapath por meio de uma FSM.

A separação entre datapath e controle deverá ser claramente apresentada no relatório.

7. Módulos Obrigatórios do Datapath

A implementação deverá conter, no mínimo, os módulos descritos a seguir.

7.1 Gerador Pseudoaleatório

Responsável pela geração dos símbolos da sequência.

Requisitos

- Implementação utilizando LFSR;
- Produção de valores entre 0 e 3;
- Operação síncrona ao clock.

Entradas

- clk
- rst
- enable

Saídas

- simbolo_randomico[1:0]

7.2 Memória da Sequência

Responsável por armazenar a sequência do jogo.

Requisitos

- Capacidade de 16 posições;
- Cada posição deve armazenar 2 bits;
- Leitura e escrita controladas pela FSM.

Entradas

- write_enable
- write_address
- write_data
- read_address

Saídas

- read_data

7.3 Registrador de Nível

Responsável por armazenar a rodada atual.

Funções

- incremento de nível;
- limpeza durante reset.

7.4 Contador de Exibição

Responsável por indicar qual elemento da sequência está sendo exibido.

Funções

- incremento;
- limpeza;
- comparação com o nível atual.

7.5 Contador de Entrada

Responsável por indicar qual posição da sequência está sendo verificada.

Funções

- incremento;
- limpeza;
- comparação com o nível atual.

7.6 Comparador de Sequência

Responsável por verificar se o símbolo informado pelo jogador corresponde ao símbolo esperado.

Entradas

- simbolo_esperado
- simbolo_jogado

Saídas

- resultado_comparacao

7.7 Temporizador

Responsável pelo controle temporal do jogo.

Funções

- tempo de exibição dos LEDs;
- intervalo entre LEDs;
- tempo máximo de resposta do jogador.

A implementação deverá utilizar contadores síncronos derivados do clock de 50 MHz da placa.

Não será permitido o uso de atrasos de simulação (#delay).

7.8 Decodificador para Display de Sete Segmentos

Responsável pela exibição:

- do nível atual;
- do estado da FSM.

7.9 Debouncer

Módulo para filtragem de ruído mecânico das entradas KEY0-KEY3, garantindo que apenas um pulso seja registrado por clique.

8. Unidade de Controle (FSM)

A unidade de controle deverá ser implementada através de uma Máquina de Estados Finitos.

A FSM será responsável pela geração de todos os sinais de controle do datapath.

9. Requisitos de Projeto

A implementação deverá:

- ser totalmente sintetizável;
- utilizar projeto modular;
- utilizar reset síncrono ou assíncrono documentado;
- separar claramente datapath e controle;
- utilizar apenas lógica desenvolvida pela equipe.

Não será permitido implementar toda a lógica do jogo em um único módulo.

10. Simulações e testbenches (usar Modelsim)

Deverão ser apresentados testbenches demonstrando:

Caso 1: Jogador completa três rodadas consecutivas.

Caso 2: Jogador vence a partida.

Caso 3: Reset durante a execução.

Caso 4: Timeout da entrada do jogador.

11. Critérios de Avaliação

Item	Peso
Projeto do Datapath	15%
Projeto da FSM	15%
Implementação Verilog	20%
Simulação e Testes	10%
Relatório	20%
Vídeo e Apresentação	20%

12. Bonificações e penalizações

Poderão receber até 10% por funcionalidade na nota, os projetos que implementarem:

- apresentação do tempo restante para o jogador nos displays HEX
- múltiplos níveis de dificuldade (tempo variável para o jogador);
- animações de vitória e derrota;
- armazenamento de recorde e visualização;
- aumentar o número de níveis suportados para até 32

Penalização de 10% por item da entrega parcial não realizado